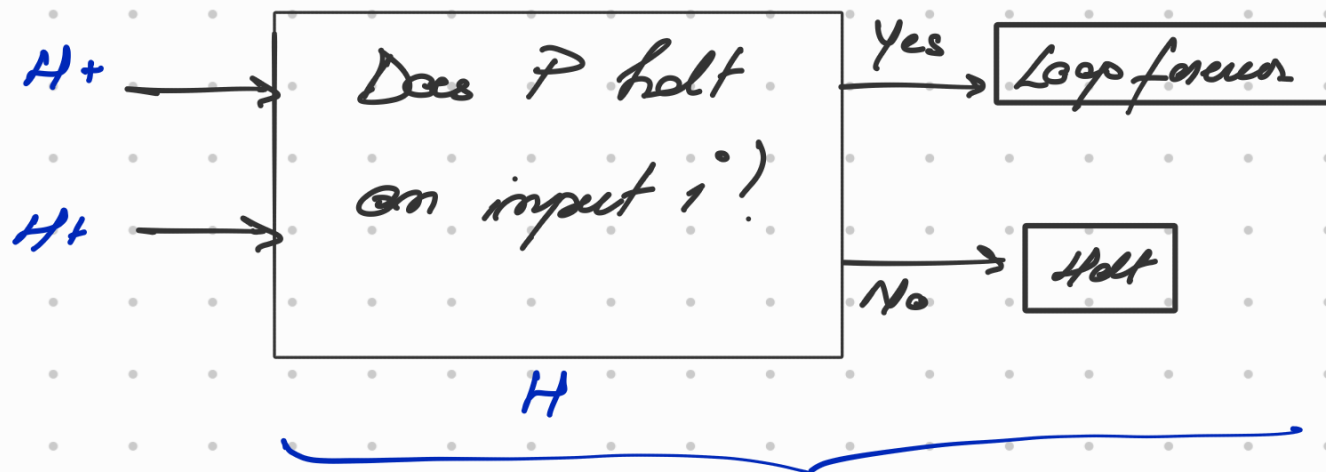


1. Computephile $\rightarrow$ Turing playlist

The halting problem:



$H+$

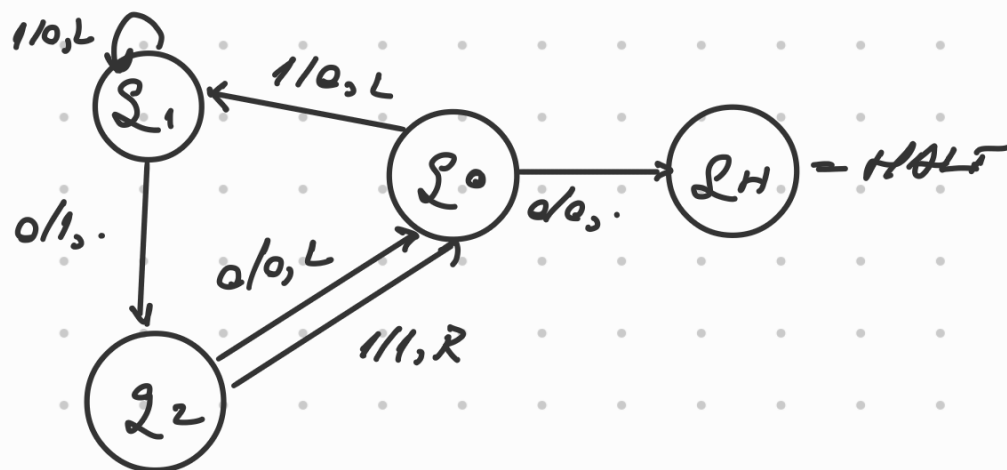
$$\left. \begin{array}{l} (H+ \text{ halts}) \rightarrow H+ \Rightarrow \text{Loops} \\ (H+ \text{ loops}) \rightarrow H+ \Rightarrow \text{Halts} \end{array} \right\} \Rightarrow \text{If}$$

Deci nu există o funcție care poate determina dacă o funcție va returna sau nu!

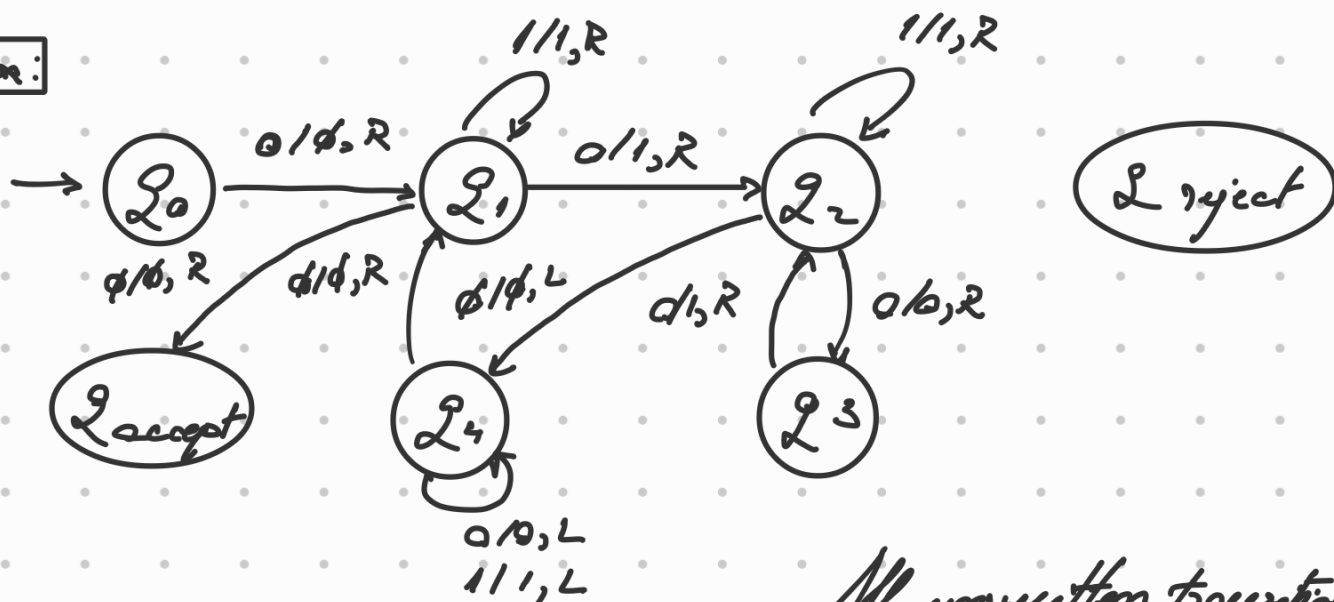
The Turing Machine:

Example:

0 1 0 1 0 1 1 0 1



Ex:



All unmentioned transitions go to Q reject.

Input:

0 0 0 0 → 0 0 0 0 → 0 1 0 0
 Q_0 Q_1 Q_2
 1

→ 0 1 0 0 → 0 1 0 1 0 → 0 1 0 1 0
 Q_3 Q_2 Q_4

→ 0 1 0 1 0 → 0 1 0 1 0 → 0 1 0 1 0
 Q_4 Q_4 Q_4

$\emptyset \mid 0 \mid \emptyset \rightarrow \emptyset \mid 1 \mid 0 \mid \emptyset \rightarrow \emptyset \mid 1 \mid 0 \mid 1 \mid \emptyset \rightarrow$
 $Q_4 \quad Q_1 \quad Q_2$
 $\rightarrow \emptyset \mid 1 \mid 1 \mid \emptyset \rightarrow \emptyset \mid 1 \mid 1 \mid 1 \mid \emptyset \rightarrow \emptyset \mid 1 \mid 1 \mid 1 \mid \emptyset \rightarrow$
 $Q_2 \quad Q_2 \quad Q_4$
 $\rightarrow \dots \emptyset \mid 1 \mid 1 \mid \emptyset \rightarrow \emptyset \mid 1 \mid 1 \mid 1 \mid \emptyset \rightarrow \dots \emptyset \mid 1 \mid 1 \mid 1 \mid \emptyset$
 $Q_4 \quad Q_1 \quad Q_1$
 $\downarrow$
 Q_{accept}

① $x == y$:

$mov \ [x], 0$
 $mov \ [y], 1$
 $mov \ R, [x]$

② If statements:

if $x == y$:

$x = 100$

← 2/sec dummy variables
 (if $x \neq y$, dummy-var = 100)

C:

$int * SELECT_x[] = \{ \&DUMMY_x, x \}$

$* SELECT_x[x == y] = 100$

SELECT-x vector cu doi pointeri:

$SELECT_x[0] = \&DUMMY_x, SELECT_x[1] = \&x$

$x == y \Rightarrow SELECT_x[1] = \&x = 100$

$x \neq y \Rightarrow \dots$

x 86:

.data

x: .long 0

dummy-x: .long 0

y: .long 0

dummy-y: .long 0

select-x: .long dummy-x, x

select-y: .long dummy-y, y

x == y:

mov x, %eax

mov \$0, (%eax)

mov y, %eax

mov \$4, (%eax)

mov x, %eax

x = 100:

mov select-x(, %eax, 1), %eax

mov \$100, (%eax)

Explicație:

select-x: +0 → adresa lui dummy-x

+4 → adresa lui x

select-y: +0 → adresa lui dummy-y

+4 → adresa lui y

Deei idustria pt. implementarea de if-uri
e cu aceste "formă selectă" (vectori de pointer)
pt. toate variabilele

↳ Varianta este de x86/x64, mai frecv
optimizată

③ Macro for equality

.macro eq dest, a, b

mul \$0, %eax

movb \a, %al

movb \$0, e(, %eax, 1)

movb \$4, e+\b

movb e(, %eax, 1), %al

movb %al, \dest

.endm

EAX = 0

AL = val. de la adresa (1B)

e[AL] = 0

e[b] = 4

AL = e[AL]

dest = AL

④ Macro for inequality

.macro neq dest, a, b,

mul \$0, %eax

movb \a, %al

movb \$4, e(, %eax, 1)

movb \$0, e+\b

movb e(, %eax, 1), %al

movb %al, \dest

.endm

Σ

Ce ne trebuie în MONTUSCATOR?

• Instructions:

- | | |
|---------|----------|
| 1. mov | 10. dec |
| 2. lea | 11. xor |
| 3. cmp | 12. push |
| 4. jmp | 13. call |
| 5. je | 14. ret |
| 6. jne | |
| 7. jle | |
| 8. jge | |
| 9. incl | |

• Operanzi:

1. registre 32b
2. memorie directă
3. memorie indirectă
4. imediat

• layout-ul memoriei

• format little endian

• convert GAS → NASM (pt. macros)

Etapă de lucru:

- ① Parser :
 - citește cod în GAS
 - recunoaște instrucțiunile
 - extrage operandii
- ② Reprezentare într-o formă constantă și predicțiile "ir" (Intermediate representation)

Practic "normalizarea" instrucțiunilor:

i.e. `ADD REG(esp), REG(ecx)`
`CMP IMM(0), REG(ecx)`
`MOV MEM(x1), REG(ecx)`

- ③ Transformare instrucțiunilor în măruri
- ④ Reducerea codului conștient în sintaxa NASM

Exercitiu pe foaie:

Ceal dot (Lab 0x02, ex 0x03):

.dote

x1: .ascii "maria"

x2: .ascii "maria1"

x3: .ascii "maria2"

.text

.global main

main:

mov \$4, %eax

mov \$1, %ebx

mov \$x1, %ecx

mov \$14, %edx

int \$0x80

} syscall 1

end

movl \$1, %eax

xor %ebx, %ebx

int \$0x80

} syscall 2

IR:

enum operand-type

{

OP_REG,

OP_IMM,

OP_LABEL,

};

```
enum reg
{
    EAX, EBX, ECX, EDX
};
```

```
struct operand
{
    enum operand-type type;
    union {
        enum reg reg;
        int imm;
        char *label;
    };
};
```

Operand

```
enum instr-type {
    i-MOV,
    i-INT,
    i-XOR
};
```

```
struct instr {
    enum instr-type type;
    struct operand dst;
    struct operand src;
};
```

Instruction

Erwiderte :

$\text{mov } \$4, \%eax \Leftrightarrow \{i_mov, REG(EAX), IMM(4)\}$

$\text{mov } \$1, \%ebx \Leftrightarrow \{i_mov, REG(EBX), IMM(1)\}$

$\text{mov } \$x1, \%ecx \Leftrightarrow \{i_mov, REG(ECX), LABEL('x1')\}$

$\text{mov } \$14, \%edx \Leftrightarrow \{i_mov, REG(EDX), IMM(14)\}$

$\text{int } \$0x80 \Leftrightarrow \{i_int, IMM(0x80)\}$

$\text{mul } \$1, \%eax \Leftrightarrow \{i_mov, REG(EAX), IMM(1)\}$

$\text{xor } \%ebx, \%ebx \Leftrightarrow \{i_xor, REG(EBX), REG(EBX)\}$

Jurnal de Lecții:

- 1) Am pornit cu ex 0x03 din Laboratorul 0x02.
Este un exercițiu simplu, doar cu mov-cui și syscall,
dar perfect pt. baza programului.
- 2) Pentru moment, focusul e doar pe semantica și
poziția caracteristicilor ce țin de memoria programului
și în noul program generat.

Prima variantă de pointer recunoscute doar:

- mov
- xor
- int

- 3) Am avut o problemă cu referințe. Date pt. cō
expresiile condiționate nu conțineau variabile "x1" →
dădea segfault. Acum cō am adăugat să testeze
secțiunea. Date separat (for the time being)

- 4) Am avut bug la pointer: scotea "int \$80" în loc de
"int \$0x80".

- 5) Prima transformare mov-only:

`xor i, ebx, i, ebx → mov $0, i, ebx`

(temporare totuși decore în marfusetă nu putem
include în 'registru' direct)

→ Așadar:

6) A dare transformare mov-only:

mov \$x, %reg $\Rightarrow$ temp: .long 0
mov \$x, temp
mov temp, %reg

de cc!

functiune veche:

mov reg, reg
mov mem, reg
mov reg, mem
* mov mem, mem

7) A treia transformare:

mov \$x1, %ecx $\Rightarrow$ mov \$x1, temp
mov temp, %ecx

8) Trebuie la ex 0x02

Tipuri de instrucțiuni:

- mov $8x$, $\%reg$
- cmp $\%reg$, $\%reg$
- jgc label
 $\nwarrow$
 " $>$ " + jmp

Următorii extindem programul cu noile tipuri de instrucțiuni, fără să le transformăm.

9) Probleme:

i) mov reusea programul (la ex anteriori avem "mov")

am transformat în mov simplu

ii) am introdus tipul INSTR_LABEL ca să salveze etichete pt. jump-uri condiționate

iii) instrucțiuni mai la IR:

jg, jmp, LABEL

10) Transformarea cuyp și jgc în mov-early

11) Instrucțiunile inst. de transformat:

→ XOR, CMP

→ ADD, SUB

→ JMP, JGE

XOR: Look-up table

CMP: tabel de comparații masiv?

ADD / SUB: înlocuim ALU cu o variantă de stocare date în memorie

"Nu putem aduce pe-se, deci trebuie să știm deja răspunsul"

12) Instrucțiunea INC:

generăm un array unde, pt. orice nr. i ,
la $v[i]$ avem $(i+1)$.

Da oam problemă cu stocarea.

Pt. număr pe 32b, $\text{len}(\text{vector}) = 2^{32} =$

≈ 4 miliarde

$4 \text{ miliarde} \times 4 \text{ bytes} \approx 16 \text{ GB}$

Am totă RAM din picó calculatorul.

Am putea folosi o logică ca la adunarea
de zecile primare; se împărțim numerele în
4 grupuri de octeți și cu flag-uri de overflow.

$$NR = [B3][B2][B1][B0]$$

Dacă B0 dă overflow, incrementăm și B1,
etc.

(Măsurcatorul lui Chris Domus face oțet pt. fiecare
bit: 32 de operații cu bit de carry)

Probleme:

1) Îmi trebuie inst. iF (dacă nu știu
se o implementez)

2) Nu știu tehnic să fac oțet

Comentariu reduc inst. inc la max 1 BYTE.

$$\begin{aligned} \Rightarrow x=0 &\rightarrow v[x]=1 \\ x=41 &\rightarrow v[x]=42 \\ x=255 &\rightarrow v[x]=0 \end{aligned}$$

Traducere în mov-only:

inc ecx = mov \$0, %ecx (pointer în array)
mov %ecx, %ecx
movl inc_table(%ecx), %ecx

"Move from Extended Byte to Long"
mov %ecx, %ecx

→ $\%ebx = \text{adresa array} + \text{valoare din } \%ebx \rightarrow \%eax$
Pe $\%eax = 41$:

1. $\%eax$ 41
2. $\%ebx$ 0
3. $\%ebx$ 41
4. $\%ebx$ $\text{inc-table}[41] = 42$
5. $\%eax$ 42

13) Instrucțiunile ADD și SUB:

Pt. adunarea a două valori, am putea replica logic într-un tabel 2D (matrice).

Pt. (add a, b), tabelul ar fi o matrice 256×256 .
Căutat în memorie. Rezultatul ar fi la

$$\text{index} = (a \times 256) + b$$

! Nu merge pt. că matricea adunare ca
să calculăm adunare. Prohibitiv !!!

Soluție:

Avem o modalitate de a face acest calcul,
fără să îl scriem pe x, în hardware.

$$\%bx = \%bh + \%bl$$

$$(16b) \quad (8b) \quad \uparrow \quad (8b)$$

concatenare

$$\text{Val } \%bx = (\text{Val } \%bh \times 256) + \text{Val } \%bl$$

add a, b = mov a, %bh ($\%bx = a \times 256$)
 mov b, %bl
 movl add_table(%ebx), %eax

Probleme:

- 1) înmulțirea nu este verificată deoarece
 e posibil ca în %ebx să am unu din greseli
 (Register Aliasing)

Du' mai întâi copii valorile reg în temp1 și
 temp2 și după cite %ebx

It. inst. SUB, logica e aceeași dar adaptată.

Tabloul de ADD e simetric ($A+B = B+A$), deci
 nu contează care e în %bh și care în %bl.

Da la SUB, ($B-A \neq A-B$).

AT&T: sub A, B : B = B - A

$$\Rightarrow \text{index} = \underbrace{(b \times 256)}_{b \rightarrow \%bh} + \underbrace{a}_{a \rightarrow \%bl}$$

14) Instruction CMP:

$$\text{CMP } A, B = \text{flag}(B - A)$$

$$(A = B) == [(B - A) = 0] \mid (A \neq B) == [(B - A) \neq 0]$$

Dev:

Paral 1: $D = B - A$ cu SUB

$$D = \text{sub_table}(B \times 256 + A)$$

Paral 2: $D == 0$?

- $\text{zero_table}(D) = 1$ dacă $D == 0$
- $\text{zero_table}(D) = 0$ dacă $D \neq 0$

$$\text{zero_table} = \overset{0}{\downarrow} 1, \overset{1}{\downarrow} 0, 0, 0, \dots, \overset{255}{\downarrow} 0$$